

Linux

Atul Kahate

Linux History

- Linux: World's most widely used kernel
- Origins in UNIX: Ken Thompson – 1969
- Richard Stallman created GNU (1984): Free software alternative to UNIX, created UNIX utilities, but kernel was missing
- Linux kernel: Created by Linus Torvalds in 1991

Linux Kernel

- Linux Kernel = Core of the Operating System
- Applications -> Kernel -> Hardware
- Manages system resources and provides services

Function	Description
Process Management	Controls running programs
Memory Management	Allocates RAM to programs
Device Management	Communicates with hardware devices
File System Management	Manages files and storage
System Calls	Interface between applications and kernel

Kernel Architecture

- Linux = **Monolithic kernel** — all core services run in kernel space, giving high performance and direct hardware access.
- Flow: **User programs** → **System calls** → **Kernel** → **Hardware**

Feature	User Space	Kernel Space
Runs	Apps (Firefox, Python)	Kernel code (scheduler, drivers)
Hardware access	None (indirect)	Full
Security	Restricted	Privileged

Linux Distribution (“Distro”)

- **Linux Distribution (Distro):** Complete operating system built around the **Linux kernel**
- Linux Distribution = **Kernel + Software + Tools**

Purpose	Example Distribution
Desktop users	Ubuntu
Enterprise servers	Red Hat Enterprise Linux
Security testing	Kali Linux
Lightweight systems	Lubuntu
Developers	Fedora

Major Distribution Families

Family	Example Distributions
Debian-based	Ubuntu, Linux Mint
Red Hat-based	Fedora, CentOS, RHEL
Arch-based	Arch Linux, Manjaro
SUSE-based	openSUSE

Ubuntu: an example - Component	Example
Kernel	Linux kernel
Desktop	GNOME
Package Manager	APT
Shell	Bash
Installer	Ubiquity

Linux Command Line

- Command line interface (CLI) is provided by a **Shell**

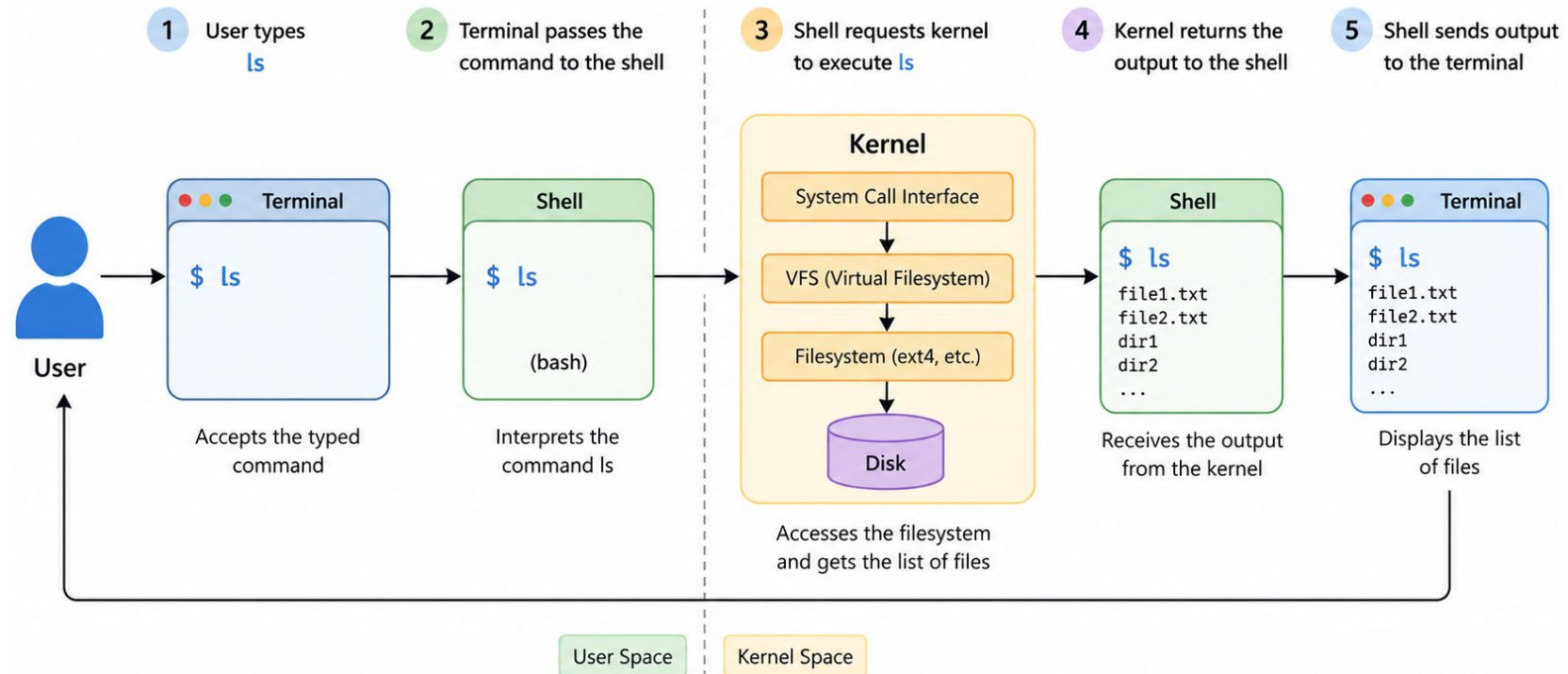
Shell	Program Name	Description
Bourne shell	sh	Original UNIX shell developed at AT&T
C shell	csh	Shell developed as part of BSD UNIX
TC shell	tcsh	Enhanced version of csh
Korn shell	ksh	Improved Bourne shell developed at AT&T
Bourne Again Shell	bash	Default shell in most Linux distributions

Various Aspects of Linux

Component	What It Is	Role in Linux	Example
Terminal	A program that provides a text interface	Lets users type commands	GNOME Terminal, xterm
Shell	A command interpreter	Reads commands and sends requests to the kernel	bash, sh, zsh
Command	Instruction given by the user	Performs a task using shell and kernel	ls, pwd, cp
Kernel	Core of the operating system	Manages hardware and system resources	Linux Kernel

How a User Command is Processed? (e.g. ls)

Component	Action
Terminal	Accepts the typed command
Shell	Interprets the command ls
Kernel	Accesses the filesystem
Terminal	Displays the list of files



The Shell Prompt

- Format: username@machine:directory\$
- Example: atul@lubuntu:~\$

Symbol	Means
\$	Normal user
#	Root user

- Summary: the prompt tells you **who** you are, **where** you are, and **what power** you have

Valid and Invalid Commands

Type	Meaning	Example
Valid	Correct syntax, runs successfully	ls, pwd, date
Invalid	Wrong syntax or unknown command → shows error	lss, password, abc123

- Common mistake: Forgetting to close a quote
 - echo "Hello C-DAC
- The shell thinks the command isn't finished (waiting for the closing "), so the prompt won't return
- Fix: Press Ctrl+C to cancel and get your prompt back

The date Command

- Show current date/time: **date**
- Custom format: **date +"%A, %d %B %y"** → *Tuesday, 17 March 26*

Specifier	Meaning	Example
%A	Full weekday name	Tuesday
%d	Day of month (01–31)	17
%B	Full month name	March
%y	Year (last 2 digits)	26

- More examples
 - **date +"%d-%m-%Y"** # 17-03-2026
 - **date +"%d %b %Y"** # 17 March 2026

The Manual Page

- The **man** command
 - Provides documentation/help for Linux commands
 - Shows detailed information about command usage
- Syntax: man command
- Example: man sort
- q to quit

The which Command

- The **which** command
 - Finds the location (path) of a command's executable file
 - Searches in directories listed in the PATH variable
- Syntax: which command
- Examples
 - which clear
 - which python

Linux Editors

Editor	Description	Key Features	Installation	Start Command
vi	Original Linux editor	Lightweight, available on all systems	Pre-installed	vi filename
vim	Improved version of vi	Syntax highlighting, better usability	Usually pre-installed / sudo apt install vim	vim filename
emacs	Highly customizable editor	Can be configured for different programming languages, extensible	sudo apt install emacs	emacs filename
nano	Simple and user-friendly editor	Easy for beginners, on-screen shortcuts	sudo apt-get update sudo apt-get install nano	nano filename
VS Code	Modern GUI-based editor	Extensions, debugging, Git integration	sudo snap install --classic code	code

vi: Create a Document

Step	Action	Keys / Command
1	Open/Create file	vi myfile.txt
2	Enter insert mode	Press i
3	Type text	Hello Linux
4	Exit insert mode	Press Esc
5	Save file	:w + Enter
6	Quit editor	:q + Enter
7	(Shortcut) Save & quit	:wq + Enter

vi: Edit File Contents

Step	Action	Keys / Command
1	Open existing file	vi myfile.txt
2	Move cursor	Arrow keys
3	Enter insert mode to edit	Press i
4	Modify text	(edit as needed)
5	Exit insert mode	Press Esc
6	Delete a character	Press x
7	Delete a full line	Press dd
8	Save changes	:w + Enter
9	Quit editor	:q + Enter
10	(Shortcut) Save & quit	:wq + Enter

Basic vi Commands

Action	Keys / Command	Description
Move left	h	Move cursor left
Move right	l	Move cursor right
Move up	k	Move cursor up
Move down	j	Move cursor down
Go to beginning of line	0	Move to start of line
Go to end of line	\$	Move to end of line
Insert at cursor	i	Start typing at cursor
Insert at beginning of line	I	Start typing at start
Append after cursor	a	Insert after cursor
Open new line below	o	New line below and insert
Copy (yank) a line	yy	Copy current line
Paste	p	Paste copied text
Undo last change	u	Undo previous action
Redo (in Vim)	Ctrl + r	Redo undone action
Search word	/word	Search forward
Next search result	n	Move to next match

Linux Command Format

- General format: `command -option argument`

Component	Meaning	Example
Command	Name of the command to execute	date, ls, cp
Option	Modifies command behavior	-l, -a, --help
Argument	Input given to the command	filename, path

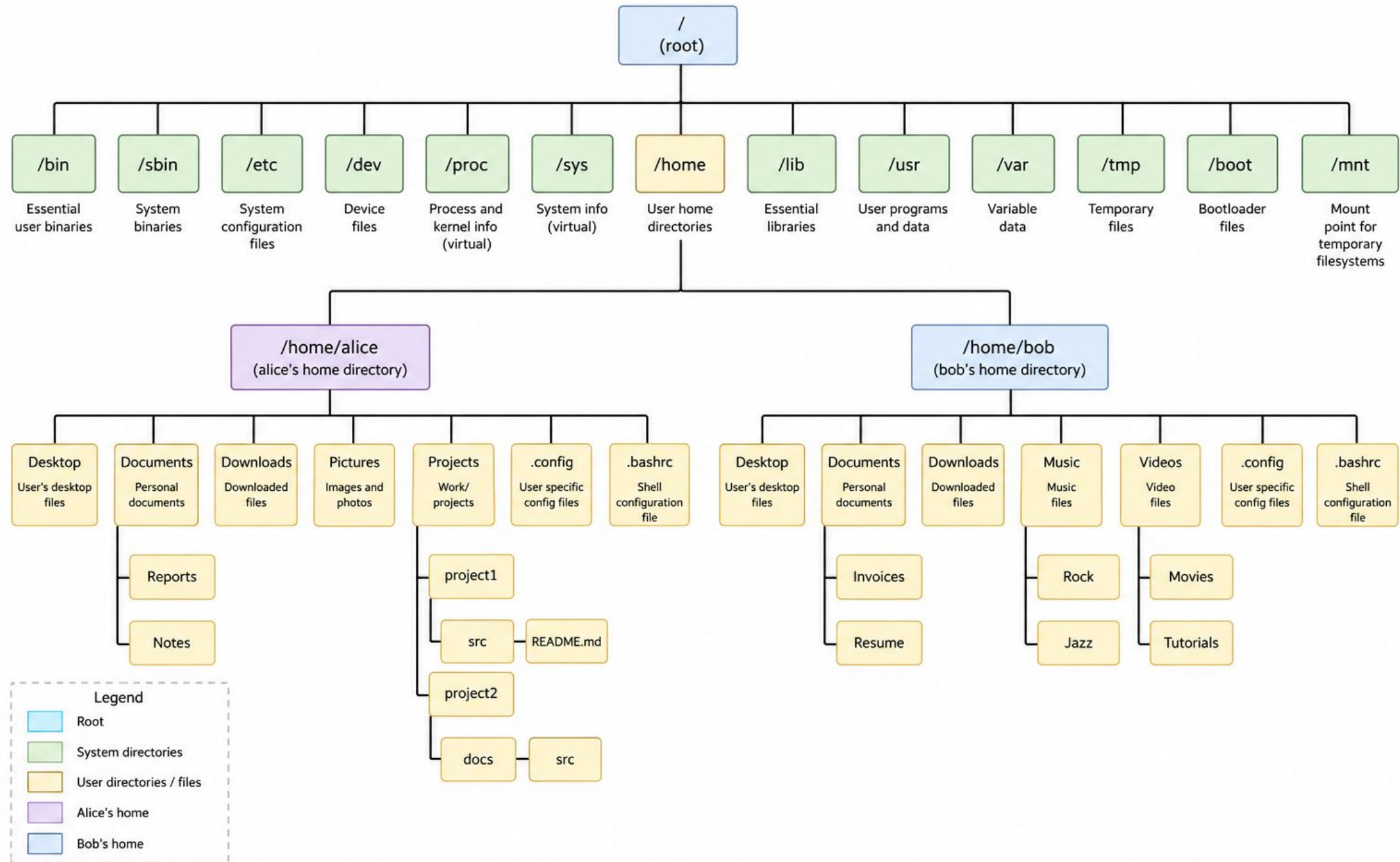
- Options can be short or long

Type	Format	Description
Short option	-l	Single hyphen, single letter
Long option	--iso-8601	Double hyphen, full name

- Examples

Command	Explanation
date -l	Displays date in ISO format
date --iso-8601	Same as above (long option)
date -r /path/to/file	Shows last modified date of a file

Linux File System Hierarchy



Linux File System

- Tree-like structure starting from the root directory (/)
- This structure ensures that users and software can reliably find files across different Linux distributions

Directory	Purpose	Key Contents
/	Root	The primary directory that contains all other directories and files.
/bin	User Binaries	Essential command binaries for all users (e.g., ls, cp, ping).
/boot	Boot Files	Static files of the boot loader, including the Linux kernel and Grub configuration.
/dev	Device Files	Representation of hardware components as files (e.g., /dev/sda for a disk).
/etc	Configuration	System-wide configuration files and startup scripts for services.
/home	User Home	Personal directories for users to store documents and settings (e.g., /home/atul).
/lib	System Libraries	Shared library files required by binaries in /bin and /sbin.
/media	Removable Media	Mount points for removable devices like USB drives or CD-ROMs.
/mnt	Mount	A temporary mount point for filesystem installations or external drives.
/opt	Optional Software	Add-on application software packages from third-party vendors.

Directory Commands

Command	Action	Example
pwd	Print Working Directory: Shows exactly where you are.	pwd
ls	List: Shows files and folders in your current spot.	ls -l (long format)
cd	Change Directory: Moves you to a new folder.	cd /home/user
mkdir	Make Directory: Creates a new folder.	mkdir new_folder
rmdir	Remove Directory: Deletes an empty folder.	rmdir old_folder
rm -r	Remove Recursive: Deletes a folder and everything inside it.	rm -r project_dir
mv	Move/Rename: Moves a folder or changes its name.	mv folder1 folder2
cp -r	Copy Recursive: Copies a folder and its entire contents.	cp -r source backup

File Creation and Viewing

Command	Purpose	Example
touch	Create empty file	touch file.txt
cat	Display file content	cat file.txt
less	View file page by page	less file.txt
head	First 10 lines	head file.txt
tail	Last 10 lines	tail file.txt

File Editing and Copying

Command	Purpose	Example
cp	Copy file	cp file.txt backup.txt
mv	Move or rename file	mv file.txt newfile.txt
vi	Edit file	vi file.txt
nano	Edit file (easy)	nano file.txt

File Deletion

Command	Purpose	Example
rm	Delete file	rm file.txt
rm -i	Delete with confirmation	rm -i file.txt
rm -r	Delete directory recursively	rm -r myfolder

File Information

Command	Purpose	Example
ls	List files	ls
ls -l	Detailed list	ls -l
stat	Detailed file info	stat file.txt
file	File type	file file.txt

Directory Operations

Command	Purpose	Example
mkdir	Create directory	mkdir mydir
rmdir	Remove empty directory	rmdir mydir
cd	Change directory	cd mydir
pwd	Show current directory	pwd

Exercise

Step	Task
1	Display current directory
2	Create a directory named lab1
3	Move into lab1
4	Create a file named file1.txt
5	Open the file and add some text
6	Display the file contents
7	Copy file1.txt to file2.txt
8	Rename file2.txt to file_renamed.txt
9	List all files
10	Display detailed file information
11	Show file type of file1.txt
12	Delete file_renamed.txt
13	Go back to parent directory
14	Delete the directory lab1

Step	Task	Command
1	Display current directory	pwd
2	Create a directory named lab1	mkdir lab1
3	Move into lab1	cd lab1
4	Create a file named file1.txt	touch file1.txt
5	Open the file and add some text	vi file1.txt
6	Display the file contents	cat file1.txt
7	Copy file1.txt to file2.txt	cp file1.txt file2.txt
8	Rename file2.txt to file_renamed.txt	mv file2.txt file_renamed.txt
9	List all files	ls
10	Display detailed file information	ls -l
11	Show file type of file1.txt	file file1.txt
12	Delete file_renamed.txt	rm file_renamed.txt
13	Go back to parent directory	cd ..
14	Delete the directory lab1	rm -r lab1

Absolute and Relative Path

Feature	Absolute Path	Relative Path
Definition	Full path from root directory	Path relative to current directory
Starts with	/ (root)	Does not start with /
Depends on current location	No	Yes
Always same?	Yes	Changes based on location
Example	/home/user/docs/file.txt	docs/file.txt
Another Example	/etc/passwd	../file.txt

Absolute and Relative Path Examples

- Absolute path
 - `cd /home/user/docs`
- Relative path
 - `cd docs`
 - `cd ../docs`

Special Symbols in Linux Paths

Symbol	Meaning	Example	Explanation
.	Current directory	./file.txt	Refers to current location
..	Parent directory	../file.txt	One level up
/	Root directory	/home/user	Top of the filesystem
~	Home directory	~/file.txt	User's home folder

```
cd .      # Stay in current directory
cd ..     # Go to parent directory
cd /      # Go to root directory
cd ~      # Go to home directory
cd ~/docs # Go to docs inside home
```

Exercises

No.	Task
1	Stay in the current directory
2	Move one level up from current directory
3	Go to the root directory from anywhere
4	Go to your home directory and create a docs directory
5	From any location, go to docs folder inside home directory
6	From /home/user/docs, go to /home/user
7	From /home/user/docs, go to /home
8	From /home/user/docs, access file.txt in the same directory using relative path
9	From /home/user/docs, access file.txt in parent directory
10	From /home/user/docs, go to root and then to /home

Solutions

No.	Task	Command
1	Stay in the current directory	cd .
2	Move one level up from current directory	cd ..
3	Go to the root directory from anywhere	cd /
4	Go to your home directory and create a docs directory	cd ~ mkdir docs
5	From any location, go to docs folder inside home directory	cd ~/docs
6	From /home/user/docs, go to /home/user	cd ..
7	From /home/user/docs, go to /home	cd ../..
8	From /home/user/docs, access file.txt in the same directory using relative path	cat ./file.txt
9	From /home/user/docs, access file.txt in parent directory	cat ../file.txt
10	From /home/user/docs, go to root and then to /home	cd /home

Superuser Commands

- **su** = become someone else, **sudo** = borrow the power of the superuser for a command

Feature	su	sudo
What it does	Switch to another user (usually root)	Run one command as another user
Password	Target user's password	Your own password
Duration	Stays logged in as that user	Only for that one command
Security	Less secure (shares root password)	More secure (logs who did what)
Setup	Works by default	Needs to be added to sudo/wheel group

How to Become a Root User?

- In a standard Ubuntu installation, the **root** user account is locked by default for security
- Result: There is no root password to enter, so you cannot log in directly as root
- Solution: Open an interactive shell
 - **sudo -i**
 - This simulates a root login, loading the root user's environment variables and changing your directory to /root
 - Your prompt will typically change from \$ to #
 - To exit, type *exit* and ENTER key, or use Ctrl+D
 - Alternate command: **sudo su -**

Comparison Table

Method	Command	Best For
Standard Sudo	<code>sudo <cmd></code>	One-off tasks (installs, updates)
Login Shell	<code>sudo -i</code>	Heavy configuration, multi-step work
Substitute Shell	<code>sudo su</code>	Quick tasks needing root for a few minutes

User Creation/Modification Commands

Task	Command	Description
Create User (Simple)	<code>sudo useradd [username]</code>	Creates a user with default settings (no home directory by default)
Create User (Interactive)	<code>sudo adduser [username]</code>	A friendly script that prompts for password, name, and creates /home – Debian/Ubuntu only
Set/Change Password	<code>sudo passwd [username]</code>	Assigns or updates the password for the specified user
Modify User	<code>sudo usermod -aG [group] [user]</code>	Adds a user to a supplementary group (e.g., adding to sudo)
Delete User	<code>sudo userdel -r [username]</code>	Removes the user. The -r flag also deletes their home directory
Check User Info	<code>id [username]</code>	Displays the UID, GID, and groups the user belongs to

Adding Users to Sudoers Group

- Method 1: Add user to sudoers group directly

sudo usermod -aG sudo username

- Method 2: Edit the /etc/sudoers file

sudo visudo

- Then add this line to the opened file

username ALL=(ALL:ALL) ALL

Group Command

- Group: Allows us to manage permissions for multiple users simultaneously

Task	Command	Description
Create Group	<code>sudo groupadd [groupname]</code>	Creates a new group.
Delete Group	<code>sudo groupdel [groupname]</code>	Removes an existing group.
List Groups	<code>groups [username]</code>	Shows all groups a specific user is a member of.
List All Groups	<code>getent group</code>	Displays a list of all groups defined on the system.

Command	Action	Description
<code>sudo usermod -aG marketing john</code>	Add user to group	Adds the user john to the marketing group without removing them from existing groups.
<code>sudo gpasswd -d john marketing</code>	Remove user from group	Removes the user john from the marketing group.

Key Files for User Information

File Path	Purpose	Key Fields (Separated by :)
/etc/passwd	User Account Info	Username, Password (usually x), UID, GID, User Info, Home Directory, Default Shell.
/etc/shadow	Hashed Passwords	Username, Hashed Password, Last Change Date, Minimum/Maximum Password Age, Expiration.
/etc/group	Group Definitions	Group Name, Group Password (rarely used), GID, List of Users in the group.
/etc/gshadow	Group Passwords	Secure group information (similar to /etc/shadow but for groups).
/etc/sudoers	Sudo Privileges	Controls which users or groups can run commands as root (managed sudo visudo).

Exercise

#	Task Description
1	Create a new user named student1 with a default home directory.
2	Set or change the password for the user student1.
3	Create a new group named developers.
4	Add the existing user student1 to the developers group as a secondary group.
5	Change the login name of student1 to intern1 keeping the home directory unchanged.
6	Delete the user intern1 and remove their home directory simultaneously.
7	Lock a user's account (e.g., user2) to prevent login.
8	Unlock a previously locked user account (e.g., user2).
9	Display all groups that the current user belongs to.
10	Change the primary group of user2 to developers.
11	Create a user tempuser with an account expiry date of 2026-12-31.
12	View the password aging/expiry details for a specific user.
13	Remove a user named user2 from the group developers without deleting the user.
14	Delete the group named developers from the system.
15	Find the UID and GID information for a specific user.

Solutions

#	Task Description	Solution
1	Create a new user named student1 with a default home directory.	sudo useradd -m student1
2	Set or change the password for the user student1.	sudo passwd student1
3	Create a new system group named developers.	sudo groupadd developers
4	Add the existing user student1 to the developers group as a secondary group.	sudo usermod -aG developers student1
5	Change the login name of student1 to intern1.	sudo usermod -l intern1 student1
6	Delete the user intern1 and remove their home directory simultaneously.	sudo userdel -r intern1
7	Lock a user's account (e.g., user2) to prevent login.	sudo usermod -L user2
8	Unlock a previously locked user account (e.g., user2).	sudo usermod -U user2
9	Display all groups that the current user belongs to.	groups
10	Change the primary group of user2 to developers.	sudo usermod -g developers user2
11	Create a user tempuser with an account expiry date of 2026-12-31.	sudo useradd -e 2026-12-31 tempuser
12	View the password aging/expiry details for a specific user.	sudo chage -l user2
13	Remove a user named user2 from the group developers without deleting the user.	sudo gpasswd -d user2 developers
14	Delete the group named developers from the system.	sudo groupdel developers
15	Find the UID and GID information for a specific user.	id user2

File Permissions

- Control **who can access a file** and **what they can do**

Symbol	Meaning
r	Read (view file)
w	Write (modify file)
x	Execute (run file/program)

Category	Meaning
User (u)	Owner of the file
Group (g)	Users in same group
Others (o)	Everyone else
All (a)	Everyone in the system

File Permissions

- -rwxr-xr--

Part	Meaning
-	File type
rwx	User permissions
r-x	Group permissions
r--	Others permissions

Octal File Permissions

Read	Write	Execute	Permission	Meaning	Binary	Decimal
0	0	0	- - -	No permission	000	0
0	0	1	- - x	Execute only	001	1
0	1	0	- w -	Write only	010	2
0	1	1	- w x	Write and Execute	011	3
1	0	0	r - -	Read only	100	4
1	0	1	r - x	Read and Execute	101	5
1	1	0	r w -	Read and Write	110	6
1	1	1	r w x	Read, Write, and Execute	111	7

Changing Permissions

- `chmod 755 file.txt`

Number	Permission
7	rwx
5	r-x
4	r--

- `chmod u=rwx,g=rx,o=rx file.txt`

Examples

Command	Effect
<code>chmod u+x script.sh</code>	Adds execute permission for the owner only.
<code>chmod g-w report.txt</code>	Removes write permission from the group .
<code>chmod o+r public.html</code>	Adds read permission for everyone else (others).
<code>chmod a+rw notes.txt</code>	Adds read and write permissions for all (user, group, and others).
<code>chmod ug=rwx data.sh</code>	Sets exactly read, write, and execute for user and group , ignoring previous settings.

Numeric Examples

Command	Numeric Code Breakdown	Effect
chmod 777 file.txt	7(rwx) 7(rwx) 7(rwx)	Full access for everyone (highly insecure).
chmod 755 script.sh	7(rwx) 5(r-x) 5(r-x)	Owner can do everything; others can read and execute (standard for scripts).
chmod 644 config.conf	6(rw-) 4(r--) 4(r--)	Owner can read/write ; others can only read (standard for docs).
chmod 600 private.key	6(rw-) 0(---) 0(---)	Only the owner can read and write; no one else has access.
chmod 700 secret.sh	7(rwx) 0(---) 0(---)	Only the owner can read, write, and execute.

Exercises

No.	Task
1	Give read, write, execute permission to owner only for file.txt
2	Give read permission to everyone for file.txt
3	Remove write permission from group for file.txt
4	Set permissions to rwxr-xr-x for file.txt using numeric mode
5	Set permissions to rw-r--r-- for file.txt using numeric mode
6	Add execute permission for owner in script.sh
7	Remove all permissions for others in file.txt
8	Give full permissions to all users for file.txt
9	Set permissions to read and write for owner, read for group and others
10	Remove execute permission for all users from file.txt
11	Add read permission for group in file.txt
12	Remove all permissions from file.txt
13	Set permissions to rwx----- using numeric mode
14	Add execute permission for all users to script.sh
15	Change permissions of all .sh files to executable in current directory

No.	Task	Command
1	Give read, write, execute permission to owner only for file.txt	chmod 700 file.txt
2	Give read permission to everyone for file.txt	chmod a+r file.txt
3	Remove write permission from group for file.txt	chmod g-w file.txt
4	Set permissions to rwxr-xr-x for file.txt using numeric mode	chmod 755 file.txt
5	Set permissions to rw-r--r-- for file.txt using numeric mode	chmod 644 file.txt
6	Add execute permission for owner in script.sh	chmod u+x script.sh
7	Remove all permissions for others in file.txt	chmod o-rwx file.txt
8	Give full permissions to all users for file.txt	chmod 777 file.txt
9	Set permissions to read and write for owner, read for group and others	chmod 644 file.txt
10	Remove execute permission for all users from file.txt	chmod a-x file.txt
11	Add read permission for group in file.txt	chmod g+r file.txt
12	Remove all permissions from file.txt	chmod 000 file.txt
13	Set permissions to rwx----- using numeric mode	chmod 700 file.txt
14	Add execute permission for all users to script.sh	chmod a+x script.sh
15	Change permissions of all .sh files to executable in current directory	chmod +x *.sh

Access Control Lists (ACL)

- ACL = Access Control List
- Provides fine-grained permissions beyond chmod
- Allows permissions for specific users and groups

Without ACL	With ACL
Only owner, group, others	Multiple users/groups
Limited control	Flexible control
chmod only	setfacl, getfacl

ACL Commands

Command	Purpose	Example
getfacl	View ACL	getfacl file.txt
setfacl -m	Set ACL	setfacl -m u:john:rwX file.txt
setfacl -x	Remove ACL	setfacl -x u:john file.txt
setfacl -b	Remove all ACLs	setfacl -b file.txt

Note: Just giving permission to user John using ACL will not allow him to access the file

Reason: John needs access to the entire path

Solution: After the above setfacl command, also run the following command:

setfacl -m u:john:x /home/atul

Then try **sudo su john** and **cat /home/atul/file.txt**, it will work

ACL Demo

- Step 1: Create a new user called alice using adduser command
- Step 2: Create a new file called share.txt and add a line in it using nano
- Step 3: Check ACL on the file: `getfacl share.txt`
- Step 4: Give permission to Alice on the file using setfacl: `setfacl -m u:alice:rw share.txt`
- Step 5: Recheck ACL on the file: `getfacl share.txt`
- Step 6: Switch to user Alice and try to access the file
 - (a) `sudo su alice`
 - (b) `cat /home/atul/share.txt`
- We will get permission denied error
- Step 7: Type exit to come back to original user
- Step 8: Add ACL on the user home directory to allow Alice access: `setfacl -m u:alice:rw /home/atul`
- Step 9: Switch to user Alice and try to access the file
 - (a) `sudo su alice`
 - (b) `cat /home/atul/share.txt`
- We should be able to view the file
- Step 10: Now try `nano /home/atul/share.txt` and edit something and save - it should allow

Shell Metacharacters

- Characters that have a special meaning
- `ls *.txt`
- `ls > output.txt`
- `ls -l | sort`
- `pwd; ls; date`
- `name="Rahul"`
- `echo $name`

Misc Commands - sort

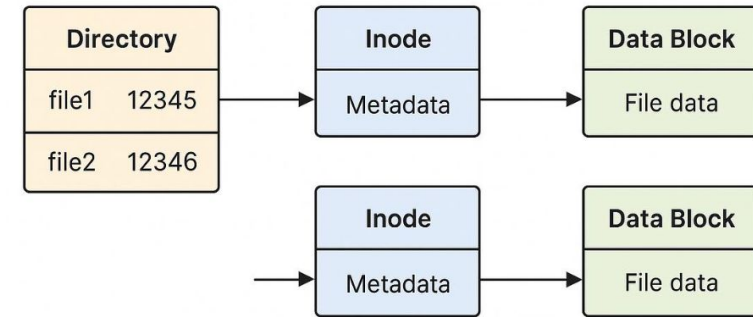
- Sorts lines of text from a file or standard input
- # Alphabetical sort
- `sort names.txt`
- # Reverse order
- `sort -r names.txt`
- # Numeric sort
- `sort -n numbers.txt`
- # Numeric sort, descending
- `sort -nr numbers.txt`
- # Remove duplicate lines while sorting
- `sort -u names.txt`
- # Sort by a specific column
- `sort -k2 data.txt`
- # Sort numerically by the second column
- `sort -k2,2n data.txt`
- # Sort human-readable sizes (KB, MB, GB)
- `sort -h sizes.txt`

Misc Commands – diff

- Compares two files line by line and shows the difference
- `diff file1.txt file2.txt`
- `diff -i file1 file2` -- Ignore case
- `diff -q file1 file2` -- Only say if the files differ

Links

- Two types: **Hard link** and **Soft link (Symbolic link)**
- **Hard link**: Same data/inode as the original file
 - In file1.txt file2.txt
 - Both names refer to the same file data
 - `ls -li` -- Both files will have the same inode number
 - If we delete the original, the link still works: `rm file1.txt` and `cat file2.txt`
- **Soft link**: Shortcut/path to a file
 - `ln -s file1.txt link.txt`
 - If the original file is deleted, symbolic link becomes a **broken/dangling link**



Global Regular Expression Print (grep) Command

- Searches for specific patterns within files or command output and displays the matching lines

Option	Description	Example
-i	Ignore Case: Searches for the pattern regardless of uppercase or lowercase.	<code>grep -i "error" log.txt</code>
-v	Invert Match: Displays all lines that do not contain the pattern.	<code>grep -v "success" log.txt</code>
-r	Recursive: Searches for the pattern in all files within a directory and its subdirectories.	<code>grep -r "function" ./src</code>
-n	Line Number: Displays the line number where the match was found.	<code>grep -n "main" script.py</code>
-c	Count: Returns the total number of lines that match the pattern instead of the text itself.	<code>grep -c "failed" auth.log</code>
-l	File Names Only: Lists only the names of files that contain the match.	<code>grep -l "TODO" *.md</code>
-w	Whole Word: Matches the pattern only if it is a whole word (not a substring).	<code>grep -w "is" file.txt</code>
-E	Extended Regex: Allows the use of complex symbols like (OR), +, and ?.	<code>grep -E "cat dog" animals.txt</code>

Regex Using grep

Symbol	Meaning	Example
^	Matches the beginning of a line.	grep "^Start" file.txt
\$	Matches the end of a line.	grep "End\$" file.txt
.	Matches any single character .	grep "b.t" (matches bat, bet, bit)
*	Matches zero or more of the preceding character.	grep "ho*p" (matches hp, hop, hoop)
[]	Matches any character inside the brackets.	grep "[0-9]" (matches any digit)

find Command

Command	Description	Example
find /path -name file	Search by name	find /home -name file.txt
find /path -iname file	Case-insensitive search	find /home -iname file.txt
find /path -type f	Find files only	find /home -type f
find /path -type d	Find directories only	find /home -type d
find /path -size +100k	Files by size	find /home -size +100k
find /path -mtime -7	Modified in last 7 days	find /home -mtime -7
find /path -name "*.txt"	Find by extension	find /home -name "*.txt"

locate Command

Task	Command	Example	Output	Notes
Search file by name	locate filename	locate file.txt	Full path(s) of file	Fast search
Case-insensitive search	locate -i name	locate -i file	Matches File, FILE, etc.	Ignores case
Limit number of results	locate -n N name	locate -n 5 file	First 5 results	Useful for large output
Count matches	locate -c name	locate -c file	Number of matches	No file list
Exact match	locate -b name	locate -b file.txt	Exact filename match	Avoid partial matches

Differences: find and locate

Feature	find	locate
Search method	Searches actual filesystem	Uses pre-built database (index)
Speed	Slower	Very fast
Accuracy	Always up-to-date	May be outdated
Syntax complexity	More complex	Very simple
Search scope	Specific directory (e.g., /home)	Entire system
Requires database	No	Yes (updatedb)
Real-time results	Yes	No
Advanced filters	Yes (size, type, time, permissions)	Limited
Example	find /home -name file.txt	locate file.txt

The tar Command

- tar = Tape Archive
- Used to combine multiple files/folders into one file
- Does NOT compress by default

Concept	Meaning	Example	Size Reduction
Archive	Combines many files into one	data.tar	No
Compression	Reduces file size	data.gz	Yes
Archive + Compression	Both together	data.tar.gz	Yes

tar Usage

c = Create
v = Verbose
f = File name
x = Extract
z = Compress

Command	Purpose	Example
tar -cvf	Create archive	tar -cvf data.tar file1.txt dir/
tar -xvf	Extract archive	tar -xvf data.tar
tar -tf	List contents	tar -tf data.tar

Task	Command	Explanation	Output
Create compressed archive (files)	tar -czvf data.tar.gz file1.txt file2.txt	Combines + compresses files	data.tar.gz
Archive a directory	tar -czvf project.tar.gz project/	Compress entire folder (recursive)	project.tar.gz
Extract archive	tar -xzvf data.tar.gz	Extract all files	Files restored
Extract to specific folder	tar -xzvf data.tar.gz -C output/	Extract into output/ directory	Files in output/
List contents	tar -tzvf data.tar.gz	View files without extracting	File list displayed

The gunzip and gzip Commands

Command	Purpose	Syntax	Example	Input	Output	Key Behavior
gzip	Compress a file	gzip file	gzip data.txt	data.txt	data.txt.gz	Original file is removed after compression
gzip -k	Compress + keep original	gzip -k file	gzip -k data.txt	data.txt	data.txt + data.txt.gz	Keeps both files
gunzip	Decompress file	gunzip file.gz	gunzip data.txt.gz	data.txt.gz	data.txt	Compressed file is removed
gzip -d	Decompress (same as gunzip)	gzip -d file.gz	gzip -d data.txt.gz	data.txt.gz	data.txt	Alternate way to unzip
zcat	View without extracting	zcat file.gz	zcat data.txt.gz	data.txt.gz	Output on screen	No file created

The gunzip/gzip Commands

Task	Command	Explanation	Output
Compress file	gzip file.txt	Compresses file	file.txt.gz (original removed)
Decompress file	gunzip file.txt.gz	Restores original file	file.txt
Decompress (alternate)	gzip -d file.txt.gz	Same as gunzip	file.txt
Keep original while compressing	gzip -k file.txt	Keeps both files	file.txt + file.txt.gz
View compressed content	zcat file.txt.gz	Displays content without extracting	Output on screen

The zip-unzip Commands

Command	Purpose	Syntax	Example	Input	Output	Key Behavior
zip	Compress files into archive	zip archive.zip files	zip data.zip file1.txt file2.txt	Files/folders	data.zip	Original files are kept
zip -r	Compress directory	zip -r archive.zip dir/	zip -r project.zip project/	Folder	project.zip	Includes subfolders
unzip	Extract archive	unzip archive.zip	unzip data.zip	.zip file	Files restored	Extracts in current directory
unzip -d	Extract to folder	unzip archive.zip -d dir/	unzip data.zip -d output/	.zip file	Files in output/	Custom location
unzip -l	List contents	unzip -l archive.zip	unzip -l data.zip	.zip file	File list	No extraction

Shell Scripting

- **Shell script:** A series of Linux commands in a file for task automation

- Example

```
#!/bin/bash
```

```
# This is a comment
```

```
echo "Hello World"
```

- Execute
 - Create file: **nano script.sh**
 - Add commands shown above
 - Give permission: **chmod u+x script.sh**
 - Run: **./script.sh**

Variables

- Used to store data (value)
- No data type declaration needed
- Value is treated as string by default
- Examples
 - name="John"
 - age=25

 - echo \$name
 - echo \$age

Add Two Numbers

- `#!/bin/bash`
- `a=10`
- `b=20`
- `sum=$((a + b))`
- `echo "Sum is $sum"`

User Input

- `#!/bin/bash`
- `read -p "Enter your name: " name`
- `echo "Hello $name"`

- `read -p "Enter two numbers: " a b`
- `echo "A = $a, B = $b"`

- `# See the impact of removing -p option, user does not know what to enter`
- `read n`
- `echo $n`

User Input

- `#!/bin/bash`
- `read -p "Enter two numbers: " x y`
- `sum=$((x + y))`
- `echo "Sum = $sum"`

Arithmetic Using Variables

- `#!/bin/bash`
- `read -p "Enter two numbers: " a b`
- `sum=$((a + b))`
- `prod=$((a * b))`
- `echo "Sum = $sum"`
- `echo "Product = $prod"`

Make New Scripts Executable: Easy Method

- Create this file (template.sh) with a single line and save it
 - `#!/bin/bash`
- Come back to shell and type: **`chmod u+x template.sh`**
- Create new shell scripts using this: **`cp template.sh my_script.sh`**
- New script is automatically executable and has the shebang operator

Floating Point Arithmetic With Hardcoded Inputs

- `#!/bin/bash`
- `a=5.5`
- `b=2.2`
- `# With scale (How many floating point digits?)`
- `sum=$(echo "scale=2; $a + $b" | bc)`
- `prod=$(echo "scale=4; $a * $b" | bc)`
- `echo "A = $a, B = $b"`
- `echo "Sum = $sum"`
- `echo "Product = $prod"`

Floating Point Arithmetic: Taking User Input

- `#!/bin/bash`
- `read -p "Enter two numbers: " a b`
- `sum=$(echo "scale=2; $a + $b" | bc)`
- `prod=$(echo "scale=2; $a * $b" | bc)`
- `echo "Sum = $sum"`
- `echo "Product = $prod"`

Exercises

No.	Exercise
1	Accept a user's name and age, store in variables, and display: Hello <name>, you are <age> years old.
2	Accept two integers from the user and display their sum, difference, and product .
3	Accept two numbers and calculate division using bc (show result up to 2 decimal places).
4	Accept radius of a circle and calculate area using bc ($\pi \approx 3.14$).
5	Accept temperature in Celsius and convert it to Fahrenheit using bc ($F = (9/5 \times C) + 32$)
6	Accept length and breadth of a rectangle and calculate area and perimeter . Area = Length x Breadth, Perimeter = 2 x (Length + Breadth)
7	Accept 3 subject marks and calculate total and average (use bc for average) .
8	Accept a number and calculate its square and cube .
9	Accept principal, rate, and time, and calculate simple interest using bc . Simple interest = $P * N * R$

Solutions

- `#!/bin/bash`
- `echo "----- Exercise 1: Name and Age -----"`
- `read -p "Enter your name: " name`
- `read -p "Enter your age: " age`
- `echo "Hello $name, you are $age years old."`
- `echo`
- `echo "----- Exercise 2: Basic Arithmetic -----"`
- `read -p "Enter two integers: " a b`
- `sum=$((a + b))`
- `diff=$((a - b))`
- `prod=$((a * b))`
- `echo "Sum = $sum"`
- `echo "Difference = $diff"`
- `echo "Product = $prod"`
- `echo`
- `echo "----- Exercise 3: Division using bc -----"`
- `read -p "Enter two numbers: " x y`
- `div=$(echo "scale=2; $x / $y" | bc)`
- `echo "Division = $div"`
- `echo`
- `echo "----- Exercise 4: Area of Circle -----"`

Simple Interest Calculation with an if Condition

- `#!/bin/bash`
- `# Simple interest calculator`
- `# p = principal amount`
- `# i = interest`
- `# t = term`
- `# r = rate of interest`
- `read -p "Enter principal, term, rate of interest separated by space: " p t r`
- `interest=$(echo "scale=2; $p * $t * $r / 100" | bc)`
- `echo "Interest is $interest"`
- `if [ $(echo "$interest > 100" | bc) -eq 1 ]; then`
- `echo "You are rich"`
- `else`
- `echo "You are poor"`
- `fi`

Swap Two Numbers

- `#!/bin/bash`
- `read -p "Enter two numbers: " a b`
- `temp=$a`
- `a=$b`
- `b=$temp`
- `echo "After swap: A = $a, B = $b"`

Area of Rectangle

- `#!/bin/bash`
- `read -p "Enter length and width: " l w`
- `area=$((l * w))`
- `echo "Area = $area"`

Temperature Conversion

- `#!/bin/bash`
- `read -p "Enter temperature in Celsius: " c`
- `f=$(( (c * 9/5) + 32 ))`
- `echo "Fahrenheit = $f"`

Temperature Conversion using Bash Calculator

- `#!/bin/bash`
- `read -p "Enter temperature in Celsius: " c`
- `f=$(echo "scale=2; ($c * 9/5) + 32" | bc)`
- `echo "Fahrenheit = $f"`
- `#####`
- `#!/bin/bash`
- `read -p "Enter temperature in Fahrenheit: " f`
- `c=$(echo "scale=2; ($f - 32) * 5/9" | bc)`
- `echo "Celsius = $c"`

Average Calculation

- Accept marks of a student in 3 subjects and display total marks and average marks
- `#!/bin/bash`
- `read -p "Enter marks for Subject 1: " m1`
- `read -p "Enter marks for Subject 2: " m2`
- `read -p "Enter marks for Subject 3: " m3`
- `# Calculate total marks`
- `total=$(echo "$m1 + $m2 + $m3" | bc)`
- `# Calculate average marks with 2 decimal places`
- `avg=$(echo "scale=2; $total / 3" | bc)`
- `echo "Total Marks = $total"`
- `echo "Average Marks = $avg"`

Result

- Modify to decide if the student passed
- `#!/bin/bash`
- `read -p "Enter marks for Subject 1: " m1`
- `read -p "Enter marks for Subject 2: " m2`
- `read -p "Enter marks for Subject 3: " m3`
- `# Calculate total marks`
- `total=$(echo "$m1 + $m2 + $m3" | bc)`
- `# Calculate average marks with 2 decimal places`
- `avg=$(echo "scale=2; $total / 3" | bc)`
- `echo "Total Marks = $total"`
- `echo "Average Marks = $avg"`
- `# Check if the student has passed (minimum 50 average)`
- `passed=$(echo "$avg >= 50" | bc)`
- `if [ "$passed" -eq 1 ]; then`
- `echo "Result: Passed"`
- `else`
- `echo "Result: Failed"`
- `fi`

Parameter Passing

- #!/bin/bash
- echo "First argument = \$1"
- To run: ./script.sh Hello

Parameter Passing

- `#!/bin/bash`
- `sum=$(( $1 + $2 ))`
- `echo "Sum = $sum"`
- To run: `./script.sh 5 10`

Count Number of Parameters/Arguments

- `#!/bin/bash`
- `echo "Total arguments = $#"`
- To run: `./script.sh a b c`

Display All Arguments

- `#!/bin/bash`
- `echo "All arguments: $@"`
- To run: `./script.sh one two three`

Script Name and Arguments

- `#!/bin/bash`
- `echo "Script name = $0"`
- `echo "First argument = $1"`
- `echo "Second argument = $2"`
- To run: `./script.sh Linux Shell`

String Length

- `#!/bin/bash`
- `read -p "Enter a string: " str`
- `length=${#str}`
- `echo "Length = $length"`

Reverse String

- `#!/bin/bash`
- `read -p "Enter a string: " str`
- `rev_str=$(echo "$str" | rev)`
- `echo "Reversed string = $rev_str"`

Count Vowels in a String

- `#!/bin/bash`
- `read -p "Enter a string: " str`
- `count=$(echo "$str" | grep -o -i "[aeiou]" | wc -l)`
- `echo "Number of vowels = $count"`

Numeric and String Relational Operators

Operator	Meaning	Example
-eq	Equal to	[\$a -eq \$b]
-ne	Not equal	[\$a -ne \$b]
-gt	Greater than	[\$a -gt \$b]
-lt	Less than	[\$a -lt \$b]
-ge	Greater than or equal	[\$a -ge \$b]
-le	Less than or equal	[\$a -le \$b]

Operator	Meaning	Example
=	Equal	["\$a" = "\$b"]
!=	Not equal	["\$a" != "\$b"]
-z	String is empty	[-z "\$a"]
-n	String not empty	[-n "\$a"]

Numeric Comparison

- `#!/bin/bash`
- `a=10`
- `b=20`
- `if [ $a -lt $b ]`
- `then`
- `echo "a is smaller"`
- `fi`

String Comparison (Default: Case Sensitive)

- `a="hello"`
- `b="Hello"`

- `if [ "$a" = "$b" ]`
- `then`
- `echo "Equal"`
- `else`
- `echo "Not Equal"`
- `fi`

- `a="hello"`
- `b="Hello"`

- `if [ "${a,,}" = "${b,,}" ] # ignore case`
- `then`
- `echo "Equal"`
- `fi`

Conditions using if

- if [condition]
 - then
 - commands
 - else
 - commands
 - fi
-
- #!/bin/bash
 - read -p "Enter a number: " num
-
- if [$((\text{num} \% 2))$ -eq 0]
 - then
 - echo "Even number"
 - else
 - echo "Odd number"
 - fi

More if Examples

- `#!/bin/bash`
- `read -p "Enter a number: " num`
- `if [ $num -gt 0 ]`
- `then`
- `echo "Positive"`
- `elif [ $num -lt 0 ]`
- `then`
- `echo "Negative"`
- `else`
- `echo "Zero"`
- `fi`

Voting Eligibility

- `#!/bin/bash`
- `read -p "Enter your age: " age`
- `if [ $age -ge 18 ]`
- `then`
- `echo "Eligible to vote"`
- `else`
- `echo "Not eligible to vote"`
- `fi`

Check If Digit

- `#!/bin/bash`
- `read -p "Enter a character: " ch`
- `if [[ $ch =~ ^[0-9]+$ ]]`
- `then`
- `echo "It is a digit"`
- `else`
- `echo "Not a digit"`
- `fi`

Largest of Three Numbers

- `#!/bin/bash`
- `read -p "Enter three numbers: " a b c`
- `if [ $a -gt $b ]`
- `then`
- `if [ $a -gt $c ]`
- `then`
- `echo "Largest = $a"`
- `else`
- `echo "Largest = $c"`
- `fi`
- `else`
- `if [ $b -gt $c ]`
- `then`
- `echo "Largest = $b"`
- `else`
- `echo "Largest = $c"`
- `fi`
- `fi`

Same Example Using AND Condition

- `#!/bin/bash`
- `# Find largest of three numbers using logical AND`
- `read -p "Enter three numbers: " a b c`
- `if [ $a -gt $b ] && [ $a -gt $c ]; then`
- `echo "Largest = $a"`
- `elif [ $b -gt $a ] && [ $b -gt $c ]; then`
- `echo "Largest = $b"`
- `else`
- `echo "Largest = $c"`
- `fi`

Grade Classification

- `#!/bin/bash`
- `read -p "Enter marks: " m`
- `if [ $m -ge 50 ]`
- `then`
- `if [ $m -ge 75 ]`
- `then`
- `echo "Distinction"`
- `else`
- `echo "Pass"`
- `fi`
- `else`
- `echo "Fail"`
- `fi`

Student Marks and Grade Calculation

- `#!/bin/bash`
- `read -p "Enter marks in subject 1: " sub1`
- `read -p "Enter marks in subject 2: " sub2`
- `read -p "Enter marks in subject 3: " sub3`
- `if [ sub1 -ge 35 && sub2 -ge 35 && sub3 -ge 35 ]; then`
- `echo "Result: Pass"`
- `total=$(echo "$sub1 + $sub2 + $sub3" | bc)`
- `average=$(echo "scale=2; $total / 3" | bc)`
- `echo "Total Marks: $total"`
- `echo "Average Marks: $average"`
- `# Determine grade based on average using bc`
- `is_first_class=$(echo "$average >= 75" | bc)`
- `if [ "$is_first_class" -eq 1 ]; then`
- `echo "Grade: First Class"`
- `else`
- `echo "Grade: Pass"`
- `fi`
- `else`
- `echo "Result: Fail"`
- `fi`

Use of && and ||

- `#!/bin/bash`
- `read -p "Enter Username: " username`
- `read -sp "Enter Password: " password`
- `echo`
- `# Allows login if (admin credentials) OR (root credentials)`
- `if [[ ( "$username" == "admin" && "$password" == "secret123" ) || ( "$username" == "root" && "$password" == "rootpass" ) ]]; then`
- `echo "Login successful!"`
- `else`
- `echo "Invalid credentials."`
- `fi`

Even/Odd and Positive/Negative

- `#!/bin/bash`
- `read -p "Enter a number: " num`
- `if [ $num -gt 0 ]`
- `then`
- `if [ $((num % 2)) -eq 0 ]`
- `then`
- `echo "Positive Even"`
- `else`
- `echo "Positive Odd"`
- `fi`
- `else`
- `if [ $((num % 2)) -eq 0 ]`
- `then`
- `echo "Negative Even"`
- `else`
- `echo "Negative Odd"`
- `fi`
- `fi`

Login Validation

- `#!/bin/bash`
- `read -p "Enter username: " user`
- `read -p "Enter password: " pass`
- `if [ "$user" = "admin" ]`
- `then`
- `if [ "$pass" = "1234" ]`
- `then`
- `echo "Login successful"`
- `else`
- `echo "Wrong password"`
- `fi`
- `else`
- `echo "Invalid username"`
- `fi`

Using AND

- `#!/bin/bash`
- `read -p "Enter three numbers: " a b c`
- `if [ $a -gt $b ] && [ $a -gt $c ]`
- `then`
- `echo "Largest = $a"`
- `elif [ $b -gt $a ] && [ $b -gt $c ]`
- `then`
- `echo "Largest = $b"`
- `else`
- `echo "Largest = $c"`
- `fi`

Login Validation Using AND

- `#!/bin/bash`
- `read -p "Enter username: " user`
- `read -p "Enter password: " pass`
- `if [ "$user" = "admin" ] && [ "$pass" = "1234" ]`
- `then`
- `echo "Login successful"`
- `else`
- `echo "Invalid username or password"`
- `fi`

OR

- `#!/bin/bash`
- `read -p "Enter marks for 2 subjects: " m1 m2`
- `if [ $m1 -ge 40 ] || [ $m2 -ge 40 ]`
- `then`
- `echo "Pass in at least one subject"`
- `else`
- `echo "Fail in both"`
- `fi`

Combining AND and OR

- `#!/bin/bash`
- `read -p "Enter username: " user`
- `read -p "Enter password: " pass`
- `if { [ "$user" = "admin" ] && [ "$pass" = "1234" ]; } || \`
- `{ [ "$user" = "root" ] && [ "$pass" = "root" ]; }`
- `then`
- `echo "Login successful"`
- `else`
- `echo "Invalid username or password"`
- `fi`

OR with Strings

- `day="Sunday"`
- `if [ "$day" = "Saturday" ] || [ "$day" = "Sunday" ]`
- `then`
- `echo "Weekend"`
- `else`
- `echo "Weekday"`
- `fi`

Login Validation

- `#!/bin/bash`
- `read -p "Enter username: " user`
- `read -p "Enter password: " pass`
- `if { [ "$user" = "admin" ] && [ "$pass" = "1234" ]; } || \`
- `{ [ "$user" = "guest" ] && [ "$pass" = "guest123" ]; }`
- `then`
- `echo "Login successful"`
- `else`
- `echo "Invalid credentials"`
- `fi`

NOT

- `#!/bin/bash`
- `read -p "Enter a number: " num`
- `if [ ! $num -eq 0 ]`
- `then`
- `echo "Number is not zero"`
- `fi`

NOT for String

- a="hello"
- b="world"
- if ["\$a" != "\$b"]
- then
- echo "Strings are different"
- fi

Student Result Calculation

- `#!/bin/bash`
- `# Input marks`
- `read -p "Enter marks for 3 subjects: " m1 m2 m3`
- `# Calculate total`
- `total=$((m1 + m2 + m3))`
- `# Calculate average`
- `avg=$((total / 3))`
- `echo "Total = $total"`
- `echo "Average = $avg"`
- `# Pass/Fail condition (assume pass if avg >= 40)`
- `if [ $avg -ge 40 ]`
- `then`
- `echo "Result: Pass"`
- `else`
- `echo "Result: Fail"`
- `fi`

Handling Decimal Points

- `#!/bin/bash`
- `# Input marks`
- `read -p "Enter marks for 3 subjects: " m1 m2 m3`
- `# Calculate total`
- `total=$((m1 + m2 + m3))`
- `# Floating-point average`
- `avg=$(echo "scale=2; $total / 3" | bc)`
- `echo "Total = $total"`
- `echo "Average = $avg"`
- `# Pass/Fail (compare using bc)`
- `if [ $(echo "$avg >= 40" | bc) -eq 1 ]`
- `then`
- `echo "Result: Pass"`
- `else`
- `echo "Result: Fail"`
- `fi`

System/Shell Variables

Variable	Meaning	Example Value	Use Case
HOME	User's home directory	/home/atul	Navigate to home
USER	Current username	atul	Display logged-in user
PWD	Present working directory	/home/atul/docs	Show current location
OLDPWD	Previous directory	/home/atul	cd - uses this
PATH	Executable search path	/usr/bin:/bin	Run commands without full path
SHELL	Current shell	/bin/bash	Know shell type
HOSTNAME	System hostname	ubuntu	Identify machine
TERM	Terminal type	xterm	Terminal compatibility
LOGNAME	Login name	atul	Similar to USER
EDITOR	Default editor	vi / nano	Used by programs
LANG	Language/locale	en_US.UTF-8	Language settings

System/Shell Variables

- `#!/bin/bash`
- `if [ "$UID" -eq 0 ]; then`
- `echo "$USER is the root user."`
- `else`
- `echo "$USER is NOT the root user."`
- `fi`

List Files in Home Directory and Show Disk Usage of Current Directory

- `#!/bin/bash`
- `echo "Files in your home directory ($HOME):"`
- `ls $HOME`
- `echo "Checking disk usage for: $PWD"`
- `du -sh $PWD`

Create a Directory in the User Home Directory

- `#!/bin/bash`
- `# Check if the "my_test" directory already exists under $HOME`
- `if [ ! -d "$HOME/my_test" ]; then`
- `mkdir -p "$HOME/my_test"`
- `echo "Directory 'my_test' has been created successfully at $HOME/my_test."`
- `else`
- `echo "Directory 'my_test' already exists at $HOME/my_test."`
- `fi`

Create a File in the Directory

- `#!/bin/bash`
- `mkdir -p "$HOME/my_test"`
- `echo "Test is successful" > "$HOME/my_test/message.txt"`
- `echo "Verifying file contents ..."`
- `cat "$HOME/my_test/message.txt"`

Backup Current Directory

- `#!/bin/bash`
- `backup_name="backup_$(date +%Y%m%d).tar.gz"`
- `tar -czf $backup_name $PWD`
- `echo "Backup created: $backup_name"`
- `read -p "Press a key to continue ..." user_key`
- `echo "Now displaying the contents of the backup file"`
- `tar -tzf $backup_name`

Special Shell Variables

Variable	Meaning	Example
\$0	Script name	script.sh
\$1, \$2...	Command-line arguments	\$1 = input1
\$#	Number of arguments	2
\$@	All arguments (separately)	arg1 arg2
\$*	All arguments (as one string)	"arg1 arg2"
\$\$	Current process ID	1234
\$?	Exit status of last command	0 (success)

Sample Usage of Shell Variables

- echo \$HOME
- echo \$USER
- echo \$PATH
- echo \$\$
- echo \$?

The case Statement

- Used for multiple conditions (like switch-case in C/Java)
 - Cleaner than multiple if-elif
 - Matches a value against different patterns
-
- case variable in
 - pattern1)
 - commands ;;
 - pattern2)
 - commands ;;
 - *)
 - default commands ;;
 - esac

case Example

- `#!/bin/bash`
- `echo "Enter a number (1-3): "`
- `read num`
- `case $num in`
 - `1) echo "You selected ONE" ;;`
 - `2) echo "You selected TWO" ;;`
 - `3) echo "You selected THREE" ;;`
 - `*) echo "Invalid choice" ;;`
- `esac`

case Example With Default Input Values and Editing Enabled (-e)

- `#!/bin/bash`
- `# Read first number with default value ... -e means arrow keys, backspace etc allowed, since we are providing default input values that user may want to edit`
- `read -e -i "10" -p "Enter first number: " a`
- `echo`
- `# Read second number with default value`
- `read -e -i "5" -p "Enter second number: " b`
- `echo`
- `# Read operation with default value`
- `read -e -i "+" -p "Enter operation (+, -, *, /): " op`
- `echo`
- `# Case statement`
- `case $op in`
- `+) echo "Result = $((a + b))" ;;`
- `-) echo "Result = $((a - b))" ;;`
- `\*) echo "Result = $((a * b))" ;;`
- `/) echo "Result = $((a / b))" ;;`
- `*) echo "Invalid operation" ;;`
- `esac`

Display if Today is a Weekday or Weekend

- `#!/bin/bash`
- `# Get the current day of the week (e.g., Mon, Tue, Sat, Sun)`
- `current_day=$(date +%a)`
- `# Check the day using a case statement`
- `case "$current_day" in`
- `Sat|Sun)`
- `echo "Weekend"`
- `;;`
- `*)`
- `echo "Weekday"`
- `;;`
- `esac`

Loops

Loop	Use Case
for	Known number of iterations
while	Run till condition is true
until	Run until condition becomes true

Loop Syntaxes

```
#!/bin/bash
```

```
# Loops
```

```
for i in 1 2 3 4 5
```

```
do
```

```
    echo "Number: $i"
```

```
done
```

```
i=1
```

```
while [ $i -le 5 ]
```

```
do
```

```
    echo "Number: $i"
```

```
    ((i++))
```

```
done
```

```
i=1
```

```
until [ $i -gt 5 ]
```

```
do
```

```
    echo "Number: $i"
```

```
    ((i++))
```

```
done
```

```
# Another syntax - for loop
```

```
for (( i=1; i <= 10; i++ ))
```

```
do
```

```
    echo "The next number is $i"
```

Sum of 10 Student Marks using “for”

- `#!/bin/bash`
- `sum=0`
- `echo "Enter marks of 10 students:"`
- `for ((i=1; i<=10; i++))`
- `do`
- `read -p "Marks $i: " num`
- `sum=$((sum + num))`
- `done`
- `echo "The sum of the 10 marks is: $sum"`

Sum of 10 Student Marks using “while”

- `#!/bin/bash`
- `sum=0`
- `i=1`
- `echo "Enter marks of 10 students below ..."`
- `while (( i <= 10 ))`
- `do`
- `read -p "Marks $i: " num`
- `sum=$((sum + num))`
- `((i++))`
- `done`
- `echo "The sum of the 10 marks is: $sum"`
- `avg=$((sum / 10))`
- `echo "The average of the 10 marks is: $avg"`

Sum of 10 Student Marks using “while” and “break”

- `#!/bin/bash`
- `sum=0`
- `count=0`
- `echo "Enter marks below (enter a negative value to stop)..."`
- `while true`
- `do`
- `read -p "Marks $((count + 1)): " num`
-
- `if (( num < 0 )); then`
- `break`
- `fi`
- `sum=$((sum + num))`
- `((count++))`
- `done`
- `if (( count > 0 )); then`
- `echo "The sum of the $count marks is: $sum"`
- `avg=$((sum / count))`
- `echo "The average of the $count marks is: $avg"`
- `else`
- `echo "No valid marks were entered."`
- `fi`

Modify to Count Passed and Failed Students

- `#!/bin/bash`
- `sum=0`
- `count=0`
- `pass_count=0`
- `fail_count=0`
- `echo "Enter marks (enter a negative value to stop)..."`
- `while true`
- `do`
- `read -p "Marks $((count + 1)): " num`
-
- `if (( num < 0 )); then`
- `break`
- `fi`
- `if (( num >= 35 )); then`
- `((pass_count++))`
- `else`
- `((fail_count++))`
- `fi`
- `sum=$((sum + num))`
- `((count++))`

Modify to Find Maximum and Minimum Marks

- `#!/bin/bash`
- `sum=0`
- `count=0`
- `pass_count=0`
- `fail_count=0`
- `max=0`
- `min=0`
- `echo "Enter marks (enter a negative value to stop)..."`
- `while true`
- `do`
- `read -p "Marks $((count + 1)): " num`
-
- `if (( num < 0 )); then`
- `break`
- `fi`
- `if (( num >= 35 )); then`
- `((pass_count++))`
- `else`
- `((fail_count++))`
- `fi`

Factorial

- `#!/bin/bash`
- `read -p "Enter a non-negative integer: " num`
- `fact=1`
- `for (( i=1; i<=num; i++ ))`
- `do`
- `fact=$((fact * i))`
- `done`
- `echo "Factorial of $num = $fact"`

Reverse the Digits of an Integer

- `#!/bin/bash`
- `read -p "Enter an integer: " num`
- `rev=0`
- `temp=$num`
- `while [ $temp -ne 0 ]`
- `do`
- `remainder=$(( temp % 10 ))`
- `rev=$(( rev * 10 + remainder ))`
- `temp=$(( temp / 10 ))`
- `done`
- `echo "Reversed of $num = $rev"`

Loop Examples – Add All Even Numbers from to 100

- `sum=0`
- `for ((i=2; i<=100; i+=2))`
- `do`
- `sum=$((sum + i))`
- `done`
- `echo "Sum (for loop) = $sum"`

- `sum=0`
- `i=2`
- `while [ $i -le 100 ]`
- `do`
- `sum=$((sum + i))`
- `((i+=2))`
- `done`
- `echo "Sum (while loop) = $sum"`

- `sum=0`
- `i=2`

No.	Exercise
1	Write a shell script to print numbers from 1 to 10 using a for loop.
2	Write a shell script to print numbers from 10 to 1 using a while loop.
3	Write a shell script to display the multiplication table of 5 using a for loop.
4	Write a shell script to find the sum of numbers from 1 to 10 using a while loop.
5	Write a shell script to print all even numbers from 1 to 20 using a for loop.
6	Write a shell script to count how many numbers are there from 1 to 15 using a while loop.
7	Write a shell script to print the squares of numbers from 1 to 10 using a for loop.
8	Write a shell script to display each character of the word LINUX one by one using a while loop.
9	Write a shell script to calculate the factorial of 5 using a for loop.
10	Write a shell script to print numbers from 1 to 20 and stop when the number becomes 12 using a while loop.

Solutions

- `#!/bin/bash`
- `echo "-----"`
- `echo "Solution 1: Print 1 to 10 using for loop"`
- `echo "-----"`
- `for i in 1 2 3 4 5 6 7 8 9 10`
- `do`
- `echo $i`
- `done`
- `echo`
- `echo "-----"`
- `echo "Solution 2: Print 10 to 1 using while loop"`
- `echo "-----"`
- `i=10`
- `while [ $i -ge 1 ]`
- `do`
- `echo $i`
- `i=$((i - 1))`
- `done`
- `echo`
- `echo "-----"`
- `echo "Solution 3: Multiplication table of 5 using for loop"`
- `echo "-----"`
- `for i in 1 2 3 4 5 6 7 8 9 10`
- `do`
- `echo "5 x $i = $((5 * i))"`
- `done`
- `echo`
- `echo "-----"`
- `echo "Solution 4: Sum of numbers from 1 to 10 using while loop"`
- `echo "-----"`
- `i=1`
- `sum=0`
- `while [ $i -le 10 ]`
- `do`
- `sum=$((sum + i))`
- `i=$((i + 1))`
- `done`

Same Example, Added if

- `sum=0`
- `for ((i=1; i<=100; i++))`
- `do`
- `if ((i % 2 == 0))`
- `then`
- `sum=$((sum + i))`
- `fi`
- `done`
- `echo "Sum (for loop) = $sum"`
- `sum=0`
- `i=1`
- `while [ $i -le 100 ]`
- `do`
- `if ((i % 2 == 0))`
- `then`
- `sum=$((sum + i))`
- `fi`
- `((i++))`
- `done`
- `echo "Sum (while loop) = $sum"`

Loop Example

- `#!/bin/bash`
- `choice="y"`
- `# until loop → repeat program until user says no`
- `until [ "$choice" = "n" ]`
- `do`
- `read -p "Enter number of students: " n`
- `total=0`
- `i=1`
- `echo "Enter marks:"`
- `# for loop → input + sum`
- `for ((i=1; i<=n; i++))`
- `do`
- `read -p "Student $i: " mark`
- `total=$((total + mark))`
- `done`
- `avg=$((total / n))`
- `echo "Total = $total"`
- `echo "Average = $avg"`

Useful Scripts: Accept a String from User and Search in All Files

- `#!/bin/bash`
- `read -p "Enter word to search: " word`
- `echo "Searching for '$word' in all files..."`
- `grep -rn "$word" *`

Sort a File and Remove Duplicates

- `#!/bin/bash`
- `read -p "Enter filename: " file`
- `echo "Sorted unique contents:"`
- `sort "$file" | uniq`

Count Lines, Words, Bytes in All Files

- `#!/bin/bash`
- `echo "File-wise count (lines, words, bytes):"`
- `echo "-----"`
- `for file in *`
- `do`
- `if [ -f "$file" ]; then`
- `wc "$file"`
- `fi`
- `done`

Backup Files into a Zip Archive

- `#!/bin/bash`
- `read -p "Enter folder name to backup: " folder`
- `read -p "Enter zip file name: " zipname`
- `zip -r "$zipname.zip" "$folder"`
- `echo "Backup created: $zipname.zip"`

Check If a File Exists and Show its Permissions

- `#!/bin/bash`
- `read -p "Enter filename: " fname`
- `if [ -e "$fname" ]; then`
 - `echo "File exists"`
 - `[ -r "$fname" ] && echo "Readable"`
 - `[ -w "$fname" ] && echo "Writable"`
 - `[ -x "$fname" ] && echo "Executable"`
- `else`
 - `echo "File does not exist"`
- `fi`

Copy All Files of a Particular Extension in a Different Directory

- `#!/bin/bash`
- `mkdir -p backup`
- `for file in *.txt`
- `do`
- `if [ -f "$file" ]; then`
- `cp "$file" backup/`
- `echo "Copied: $file"`
- `fi`
- `done`
- `echo "Backup completed."`

Create a New User

- `#!/bin/bash`
- `# Check if script is run as root`
- `if [ "$EUID" -ne 0 ]; then`
- `echo "Please run as root (use sudo)"`
- `exit 1`
- `fi`
- `# Accept username`
- `read -p "Enter new username: " username`
- `# Check if user already exists`
- `if id "$username" &>/dev/null; then`
- `echo "User already exists!"`
- `else`
- `# Create user with home directory`
- `useradd -m "$username"`
- `# Set password`
- `passwd "$username"`
- `echo "User '$username' created successfully."`
- `fi`

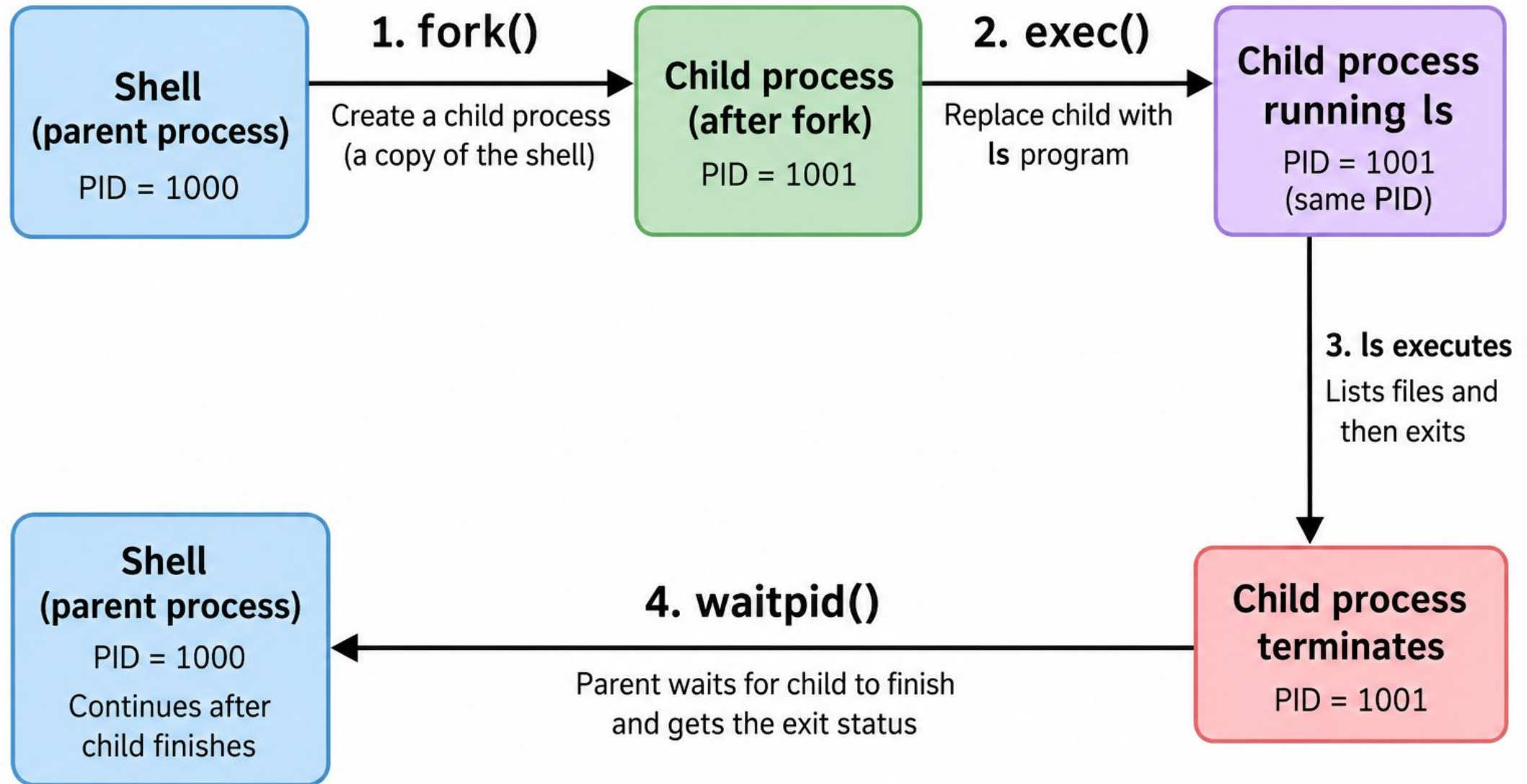
Linux Process Commands

- **Process** = A running **program**

- ps Basic view

- ps aux Detailed view

- top Live processes



Linux Processes

Type	Description
Parent	A process that creates other processes
Child	A process created by another (via fork())
Daemon	Background service (e.g., sshd, httpd)
Zombie	Finished process, but still in process table (not cleaned up)
Orphan	Child whose parent has terminated

- To see all running processes: **ps -ef | less**
- First process (init/systemd) always has a process id = 1 → adopts orphan processes
- Process creation: Parent process -> fork () -> Child process

Foreground and Background Processes

- Commands run in the foreground and occupy the terminal
- Adding & runs a command in the background
- Example: **sleep 10 &** *This is called controlling a job*

Command	Purpose
jobs	List background jobs
fg	Bring job to foreground
bg	Resume job in background
Ctrl + Z	Suspend foreground process

Foreground vs Background Execution

- `$ sleep 10`
- `#` Runs in foreground → terminal is blocked
- `$ sleep 60 &`
- `[1] 1150`
- `#` Runs in background → terminal is free, `[1]` = Job ID, `1150` = Process ID (PID)
- `$ echo "hello there"`
- `hello there`
- `#` Another foreground command runs immediately
- `$ jobs`
- `[1]+ Running sleep 60 &`
- `#` Background process is still running
- `$ fg %1`
- `sleep 60`
- `#` Bring job 1 back to foreground

Killing a Process

- **kill** <PID>, e.g. kill 1947
- **Signal**: An asynchronous message sent to a process
- Used to control process behavior (stop, terminate, pause, etc.)

Signal	Number	Purpose
SIGTERM	15	Graceful termination (default)
SIGKILL	9	Force kill (cannot be ignored)
SIGSTOP	19	Pause process
SIGCONT	18	Resume paused process

- Examples
 - kill -9 1947 # Force kill
 - kill -15 1947 # Graceful kill (default)
 - kill -STOP 1947 # Pause process
 - kill -CONT 1947 # Resume process